

Process of the programming

Peripheral required:

1. GPIOB
2. RCC_AHB1

Registers used:

1. RCC_AHB1ENR
2. GPIOB_MODER (Mode Register)
3. GPIOB_ODR (Output Data register)

RM = Reference Manual

DS = Datasheet

Project Creation:

- 1) Open STMCubeIDE.
- 2) Click on "New"
- 3) Click on "STM32 project"
- 4) In the MCU/MPU selector part enter "STM32F401CCU6"
- 5) Click on "Next".
- 6) Give the project name (e.g. STM32_Blink)
- 7) Select the Targeted project type in to "Empty".
- 8) Click on Finish.
- 9) In the Project Explorer you can see the src, click on it.
- 10) Open main.c and start writing the code.

Code Explanation :

1) At first we need the address of each peripheral that we can see in RM page 30, and to get the address of each register of that particular peripheral we add the address offset of the register with the peripheral address. For example in this case the address of RCC is 0x40023800 and we can see the offset address of the AHB1ENR is 0x30 (check RM page 118 and 137) , so actual address of the AHB1ENR will be (0x40023800 +0x30). Hence in the program we declare AHB1ENR as -

```
#define AHB1ENR (0x40023800 +0x30)
```

2) To use any peripherals we have to configure the clock related to those peripherals. In the fig.1 you can see the GPIOB is related to AHB1 so we have to enable this first. You can check this full diagram in the DS page number 14.

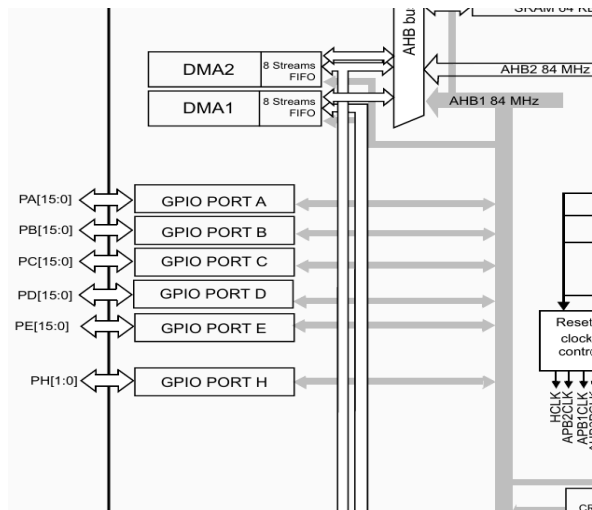


Fig.1

3) We shall enable the GPIOB clock by setting the 1st bit of the AHB1ENR to high (Check RM page no. 118). Also show in Fig.2.

6.3.9 RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2EN	DMA1EN	Reserved				
									rw	rw					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved				GPIOHEN	Reserved		GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
			rw					rw			rw	rw	rw	rw	rw

Fig.2

4) After that we shall set the GPIOB pin 12 as Output by setting 24th to 1 and 25th bit to 0 of the MODER register (Check RM page 158). It is recommended to clear these two bits first before inputting any values. Shown in fig.3.

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Fig.3

5) Now the digital output will be handled by the ODR, by setting 1 to the specific bit related to each pin to make the pin digitally high. In this case we will set the 12 to 1 to get digitally high signal, and set 0 to get digitally low signal. Show in Fig.4.

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..E and H)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y = 0..15)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..E and H).

Fig.4

6) A software busy-wait delay is generated by the for loop in side the delay function. When the function is called the loop start counting 0 to 1000000 that create a visible delay.

7) Once program writing is done check for any errors by clicking the build button in cubelIDE (short cut key ctrl + B).

8) After the click on Run -> Run as -> STM32 C/C++ application.